

```
-- =====  
-- COMPANY DATABASE STORED PROCEDURE LAB  
-- Beginner Level MySQL Stored Procedure Examples  
-- =====
```

```
-- =====  
-- STEP 1: CREATE DATABASE  
-- =====
```

```
CREATE DATABASE IF NOT EXISTS company_db;  
USE company_db;
```

```
-- =====  
-- STEP 2: CREATE EMPLOYEES TABLE  
-- =====
```

```
CREATE TABLE IF NOT EXISTS employees (  
    employee_id INT AUTO_INCREMENT PRIMARY KEY,  
    employee_name VARCHAR(100) NOT NULL,  
    department VARCHAR(100),  
    salary DECIMAL(10,2),  
    address VARCHAR(255),  
    phone_number VARCHAR(20),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
-- =====  
-- STEP 3: INSERT SAMPLE DATA  
-- =====
```

```
INSERT INTO employees  
(employee_name, department, salary, address, phone_number)  
VALUES  
( 'Aung Aung', 'IT', 800000, 'Yangon', '091111111'),  
( 'Su Su', 'HR', 600000, 'Mandalay', '092222222'),  
( 'Kyaw Kyaw', 'Finance', 900000, 'Naypyitaw', '093333333'),  
( 'Mya Mya', 'Marketing', 700000, 'Yangon', '094444444'),  
( 'Ko Ko', 'IT', 850000, 'Bago', '095555555');
```

-- =====

-- VIEW CURRENT EMPLOYEE DATA

-- =====

53 • **SELECT * FROM employees;**

Result Grid							
Filter Rows:							
Edit:   							
Export/Import:  							
Wrap Cell Content: 							
	employee_id	employee_name	department	salary	address	phone_number	created_at
▶	1	Aung Aung	IT	800000.00	Yangon	091111111	2026-05-20 14:04:29
	2	Su Su	HR	600000.00	Mandalay	092222222	2026-05-20 14:04:29
	3	Kyaw Kyaw	Finance	900000.00	Naypyitaw	093333333	2026-05-20 14:04:29
	4	Mya Mya	Marketing	700000.00	Yangon	094444444	2026-05-20 14:04:29
	5	Ko Ko	IT	850000.00	Bago	095555555	2026-05-20 14:04:29
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

-- =====

-- STORED PROCEDURE 1:

-- InsertEmployee

-- PURPOSE:

-- Insert new employee records

-- PARAMETER TYPE:

-- IN PARAMETERS

-- =====

DELIMITER \$\$

```
CREATE PROCEDURE InsertEmployee(  
    IN p_employee_name VARCHAR(100),  
    IN p_department VARCHAR(100),  
    IN p_salary DECIMAL(10,2),  
    IN p_address VARCHAR(255),  
    IN p_phone_number VARCHAR(20)  
)  
BEGIN
```

```
    INSERT INTO employees(  
        employee_name,  
        department,  
        salary,  
        address,  
        phone_number  
    )
```



```
-- =====
-- STORED PROCEDURE 2:
-- DeleteEmployee
-- PURPOSE:
-- Delete employee using employee ID
-- PARAMETER TYPE:
-- IN PARAMETER
-- =====
```

DELIMITER \$\$

CREATE PROCEDURE DeleteEmployee(

IN p_employee_id INT

)

BEGIN

DELETE FROM employees

WHERE employee_id = p_employee_id;

END \$\$

DELIMITER ;

```
-- =====
-- EXECUTE DeleteEmployee PROCEDURE
-- Delete employee with ID = 3
-- =====
```

CALL DeleteEmployee(3);

```
-- =====
-- VIEW DATA AFTER DELETE
-- =====
```

176 • **SELECT * FROM employees;**

Result Grid  Filter Rows: Edit:    Export/Import:   Wrap Cell Content: 							
	employee_id	employee_name	department	salary	address	phone_number	created_at
▶	1	Aung Aung	IT	800000.00	Yangon	091111111	2026-05-20 14:04:29
	2	Su Su	HR	600000.00	Mandalay	092222222	2026-05-20 14:04:29
	4	Mya Mya	Marketing	700000.00	Yangon	094444444	2026-05-20 14:04:29
	5	Ko Ko	IT	850000.00	Bago	095555555	2026-05-20 14:04:29
	6	Hla Hla	Admin	500000.00	Yangon	096666666	2026-05-20 14:06:00
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

```
-- =====
-- STORED PROCEDURE 3:
-- UpdateEmployeeAddress
-- PURPOSE:
-- Update employee address
-- PARAMETER TYPE:
-- IN PARAMETERS
-- =====
```

DELIMITER \$\$

```
CREATE PROCEDURE UpdateEmployeeAddress(
  IN p_employee_id INT,
  IN p_new_address VARCHAR(255)
)
BEGIN
```

```
  UPDATE employees
  SET address = p_new_address
  WHERE employee_id = p_employee_id;
```

END \$\$

DELIMITER ;

```
-- =====
-- EXECUTE UpdateEmployeeAddress PROCEDURE
-- =====
```

CALL UpdateEmployeeAddress(2, 'Taunggyi');

```
-- =====
-- VIEW DATA AFTER UPDATE
-- =====
```

219 • **SELECT * FROM employees;**

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content:							
	employee_id	employee_name	department	salary	address	phone_number	created_at
▶	1	Aung Aung	IT	800000.00	Yangon	091111111	2026-05-20 14:04:29
	2	Su Su	HR	600000.00	Taunggyi	092222222	2026-05-20 14:04:29
	4	Mya Mya	Marketing	700000.00	Yangon	094444444	2026-05-20 14:04:29
	5	Ko Ko	IT	850000.00	Bago	095555555	2026-05-20 14:04:29
	6	Hla Hla	Admin	500000.00	Yangon	096666666	2026-05-20 14:06:00
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

```
-- =====
-- STORED PROCEDURE 4:
-- GetEmployees
-- PURPOSE:
-- Retrieve all employee records
-- PARAMETER TYPE:
-- NO PARAMETERS
-- =====
```

DELIMITER \$\$

CREATE PROCEDURE GetEmployees()

BEGIN

SELECT *
FROM employees;

END \$\$

DELIMITER ;

```
-- =====
-- EXECUTE GetEmployees PROCEDURE
-- =====
```

248 **CALL** GetEmployees();

Result Grid  Filter Rows: Export:  Wrap Cell Content: 							
	employee_id	employee_name	department	salary	address	phone_number	created_at
▶	1	Aung Aung	IT	800000.00	Yangon	091111111	2026-05-20 14:04:29
	2	Su Su	HR	600000.00	Taunggyi	092222222	2026-05-20 14:04:29
	4	Mya Mya	Marketing	700000.00	Yangon	094444444	2026-05-20 14:04:29
	5	Ko Ko	IT	850000.00	Bago	095555555	2026-05-20 14:04:29
	6	Hla Hla	Admin	500000.00	Yangon	096666666	2026-05-20 14:06:00

```
-- =====
-- STORED PROCEDURE 5:
-- GetEmployeeCount
-- PURPOSE:
-- Return total employee count
-- PARAMETER TYPE:
-- OUT PARAMETER
-- =====
```

```
DELIMITER $$
CREATE PROCEDURE GetEmployeeCount(
    OUT p_total_employees INT
)
BEGIN

    SELECT COUNT(*)
    INTO p_total_employees
    FROM employees;

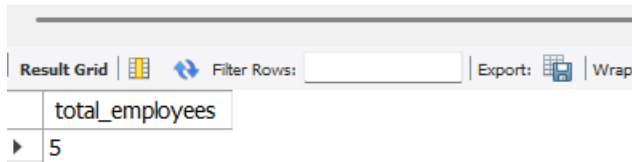
END $$
DELIMITER ;
```

```
-- =====  
-- EXECUTE GetEmployeeCount PROCEDURE  
-- =====
```

CALL GetEmployeeCount(@total);

```
-- =====  
-- DISPLAY OUTPUT VARIABLE  
-- =====
```

289 • **SELECT** @total **AS** total_employees;



The screenshot shows a SQL query result grid. The header row is labeled 'total_employees'. The first data row contains the value '5'. The interface includes a 'Result Grid' tab, a 'Filter Rows' input field, and 'Export' and 'Wrap' buttons.

total_employees
5

```
-- =====  
-- STORED PROCEDURE 6:  
-- IncreaseSalary  
-- PURPOSE:  
-- Increase employee salary and  
-- return updated salary  
-- PARAMETER TYPE:  
-- INOUT PARAMETER  
-- =====
```

DELIMITER \$\$

```
CREATE PROCEDURE IncreaseSalary(  
    IN p_employee_id INT,  
    INOUT p_increment DECIMAL(10,2)  
)  
BEGIN
```

```
    -- Increase salary  
    UPDATE employees  
    SET salary = salary + p_increment  
    WHERE employee_id = p_employee_id;
```

```
    -- Return updated salary
```



```

SELECT salary
INTO p_increment
FROM employees
WHERE employee_id = p_employee_id;

END $$
DELIMITER ;

```

```

-- =====
-- EXECUTE IncreaseSalary PROCEDURE
-- =====

```

```
SET @new_salary = 50000;
```

```
CALL IncreaseSalary(1, @new_salary);
```

```

-- =====
-- DISPLAY UPDATED SALARY
-- =====

```

347 • **SELECT** @new_salary **AS** updated_salary;

updated_salary
850000.00

```

-- =====
-- SHOW ALL STORED PROCEDURES
-- =====

```

353 • **SHOW PROCEDURE STATUS**
 354 **WHERE** Db = 'company_db';

Db	Name	Type	Definer	Modified	Created	Securi
company_db	DeleteEmployee	PROCEDURE	root@localhost	2026-05-20 14:06:46	2026-05-20 14:06:46	DEFIN
company_db	GetEmployeeCount	PROCEDURE	root@localhost	2026-05-20 14:09:44	2026-05-20 14:09:44	DEFIN
company_db	GetEmployees	PROCEDURE	root@localhost	2026-05-20 14:09:14	2026-05-20 14:09:14	DEFIN
company_db	IncreaseSalary	PROCEDURE	root@localhost	2026-05-20 14:10:34	2026-05-20 14:10:34	DEFIN
company_db	InsertEmployee	PROCEDURE	root@localhost	2026-05-20 14:05:45	2026-05-20 14:05:45	DEFIN
company_db	UpdateEmployeeAddress	PROCEDURE	root@localhost	2026-05-20 14:08:19	2026-05-20 14:08:19	DEFIN

```
-- =====
```

```
-- OPTIONAL:
```

```
-- DROP PROCEDURE EXAMPLES
```

```
-- =====
```

```
-- DROP PROCEDURE IF EXISTS InsertEmployee;
```

```
-- DROP PROCEDURE IF EXISTS DeleteEmployee;
```

```
-- DROP PROCEDURE IF EXISTS UpdateEmployeeAddress;
```

```
-- DROP PROCEDURE IF EXISTS GetEmployees;
```

```
-- DROP PROCEDURE IF EXISTS GetEmployeeCount;
```

```
-- DROP PROCEDURE IF EXISTS IncreaseSalary;
```

```
-- =====
```

```
-- END OF LAB SCRIPT
```

```
-- =====
```